SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING **Name: Thanmai Reddy**
COURSE CODE: CSA43 **Reg no:192425372**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I __T.Thanmai Reddy (192425372)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	24/25	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	24/25	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	19/20	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	14/15	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	9/10	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5/5	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

TOTAL:95/100

1. Online Symposium Portal

1. Identify Appropriate HTML5 Semantic Elements Missing in the Structure

HTML5 semantic elements provide meaningful structure to a webpage and help browsers, search engines, and assistive technologies understand its content.

The given HTML form is missing the following semantic elements:

- <header>: Defines the introductory section of the webpage, such as the symposium title.
- <main>: Represents the main content of the webpage.
- <section>: Groups related content, such as symposium registration details.
- <footer>: Contains additional information, such as copyright details and contact information.

These elements improve the organization, readability, and accessibility of the online symposium registration portal.

2. Modify the Structure to Make It Standards-Compliant

The following HTML5 code provides a standards-compliant structure for the online symposium registration portal.

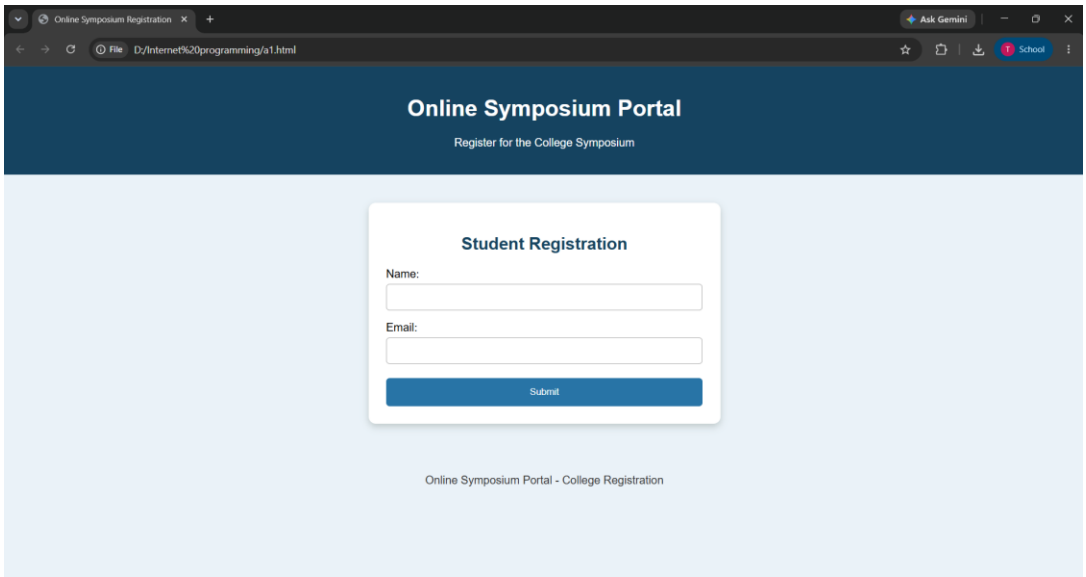
HTML Code:

```
 <!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         background-color: #eaf2f8;
         margin: 0;
         padding: 0;}
      header {
         background-color: #154360;
         color: white;
         text-align: center;
         padding: 20px; }
```

```
main {
        max-width: 450px;
        margin: 40px auto;
        padding: 25px;
        background-color: white;
        border-radius: 10px;
        box-shadow: 0 4px 10px #bfc9ca;  }
    h2 {
        color: #154360;
        text-align: center;}
    label {
        display: block;
        margin-top: 15px;
        margin-bottom: 5px;}
    input {
        width: 100%;
        padding: 10px;
        box-sizing: border-box;
        border: 1px solid #aaa;
        border-radius: 5px;  }
    button {
        width: 100%;
        margin-top: 20px;
        padding: 12px;
        background-color: #2874a6;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;  }
    button:hover {
        background-color: #154360;}
    footer {
        text-align: center;
        padding: 15px;
        color: #333; }
  </style>
</head>
<body>
  <header>
    <h1>Online Symposium Portal</h1>
```

```
    <p>Register for the College Symposium</p>
  </header>
  <main>
    <section>
      <h2>Student Registration</h2>
      <form action="/register" method="POST">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
        <button type="submit">Submit</button>
      </form>
    </section>
  </main>
  <footer>
    <p>Online Symposium Portal - College Registration</p>
  </footer>
</body>
</html>
```

Output:



Explanation:

- <!DOCTYPE html> declares the HTML5 document type.
- <html lang="en"> specifies the language of the webpage.
- <meta charset="UTF-8"> supports a wide range of characters.

- <meta name="viewport"> makes the webpage suitable for different screen sizes.
- <header>, <main>, <section>, and <footer> provide semantic structure.
- <label> elements improve accessibility by identifying form fields.
- The required attribute ensures that students enter their name and email before submitting the form.

3. Interpret the HTTP Request Generated During Form Submission

When a student fills in the registration form and clicks the Submit button, the browser generates an HTTP POST request and sends it to the server.

Explanation:

1. **POST Method:** Sends the registration information to the server for processing.
2. **Request URL:** /register identifies the server endpoint that handles registration.
3. **HTTP Version:** HTTP/1.1 specifies the protocol version.
4. **Host Header:** Identifies the server receiving the request.
5. **Content-Type:** Specifies the format of the submitted form data.
6. **Request Body:** Contains the student's name and email address.

The POST method sends form data in the request body rather than appending it to the URL. However, POST alone does not encrypt the data; HTTPS should be used to protect it during transmission.

4. Analyze the HTTP Response Cycle from Server to Browser

The HTTP response cycle describes how the server processes the registration request and returns a response to the browser.

Steps in the HTTP Response Cycle:

1. **Form Submission:** The student enters the required details and clicks Submit.
2. **HTTP Request:** The browser sends a POST request to the /register endpoint.
3. **Server Processing:** The server receives the request and validates the student's information.
4. **Database Operation:** If the information is valid, the server may store the registration details in a database.
5. **HTTP Response:** The server returns a response containing a status code, headers, and optionally a response body.
6. **Browser Display:** The browser displays the response, such as a registration confirmation .

5. Suggest Improvements for Usability and Accessibility

The following improvements can make the online symposium registration portal more user-friendly and accessible.

1. Use Labels: Provide descriptive labels for the name and email fields so users can identify the required information.
2. Add Input Validation: Use attributes such as required and type="email" to validate basic input before submission.
3. Provide Error Messages: Display clear messages when the user enters invalid information or leaves a required field empty.
4. Improve Keyboard Navigation: Ensure that users can navigate through the form using the keyboard.
5. Use Responsive Design: Apply CSS media queries and flexible layouts to support desktops, tablets, and mobile devices.
6. Ensure Security: Use HTTPS, validate input on the server, and protect stored personal information.
7. Provide Confirmation: Display a success message after registration is completed.
8. Improve Visual Design: Use readable fonts, sufficient color contrast, and appropriate spacing between form fields.

2. Cross-Browser Compatibility Issue

1. Identify Improper Nesting and Missing Closing Tags

The given HTML code contains several errors that may affect webpage structure and consistency.

- The <h1> tag is not closed with </h1>.
- The tag is not closed with .
- The document is missing the <!DOCTYPE html> declaration.
- The <html> tag does not specify the document language using the lang attribute.
- The HTML document lacks proper semantic elements for organizing the content.

These errors should be corrected to ensure that the webpage follows HTML5 standards.

2**. Analyze How Different Browsers May Interpret This Structure**

Different browsers may interpret invalid HTML differently. Some browsers automatically insert missing

closing tags or adjust the document structure, while others may display unexpected spacing or formatting.

For example, the missing </h1> tag may cause the browser to interpret the following text as part of the heading. This can lead to inconsistent text sizes and layouts across browsers.

Using valid HTML ensures more predictable rendering and improves website reliability.

3. Identify Missing Semantic Elements

HTML5 semantic elements provide meaningful structure to webpages and improve accessibility.

The following elements are suitable for the company website:

- <header>: Contains the main heading or introductory content.
- <main>: Represents the primary content of the webpage.
- <section>: Groups related content, such as the About Us section.
- <footer>: Contains copyright information or contact details.

Using these elements makes the webpage easier to understand for browsers, developers, and assistive technologies.

4. Provide a Corrected W3C-Compliant HTML Structure

The following code corrects the errors and uses appropriate HTML5 semantic elements.

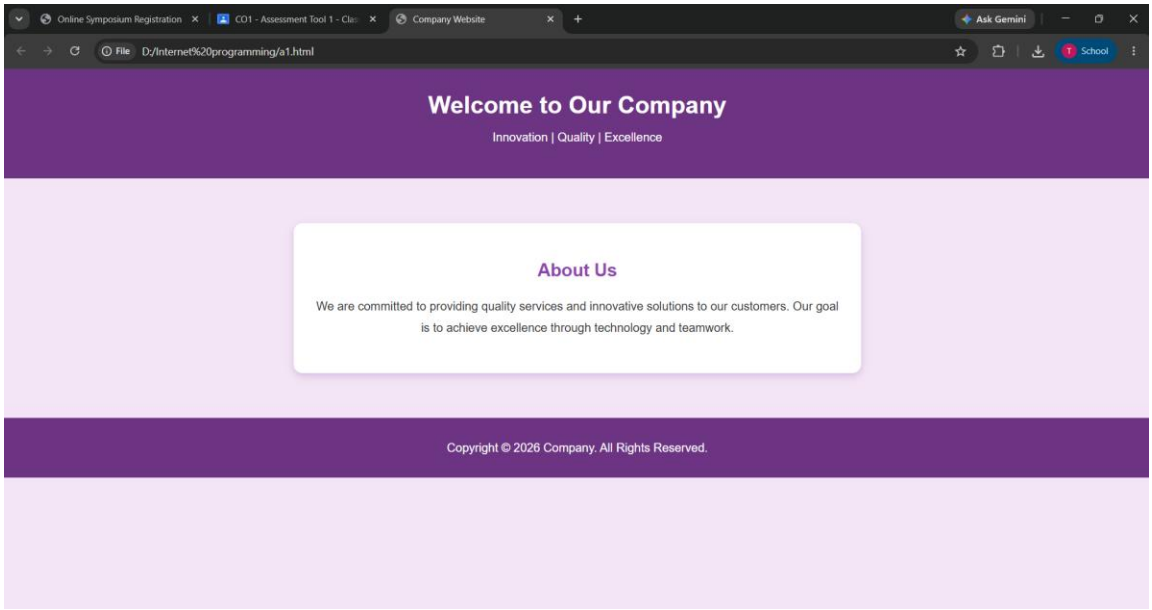
HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company Website</title>
  <style>
    * {
      box-sizing: border-box;}
    body {
      font-family: Arial, sans-serif;
      margin: 0;
```

```
      background-color: #f3e5f5;
    color: #333;  }
header {
    background-color: #6c3483;
    color: white;
    text-align: center;
    padding: 30px; }
header h1 {
    margin: 0;
    font-size: 32px;}
main {
    max-width: 800px;
    margin: 40px auto;
    padding: 20px;}
section {
    background-color: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 4px 10px #d7bde2;
    text-align: center;  }
section h2 {
    color: #8e44ad; }
section p {
    font-size: 16px;
    line-height: 1.8;   }
footer {
    background-color: #6c3483;
    color: white;
    text-align: center;
    padding: 15px;
    margin-top: 40px; }
```

```
        @media (max-width: 600px) {
      header h1 {
        font-size: 25px; }
      main {
        margin: 20px auto;
        padding: 15px;   }  }
  </style>
</head>
<body>
  <header>
    <h1>Welcome to Our Company</h1>
    <p>Innovation | Quality | Excellence</p>
  </header>
  <main>
    <section>
      <h2>About Us</h2>
      <p>
        We are committed to providing quality services
        and innovative solutions to our customers.
        Our goal is to achieve excellence through
        technology and teamwork.
      </p>
    </section>
  </main>
  <footer>
    <p>Copyright &copy; 2026 Company. All Rights Reserved.</p>
  </footer>
</body>
</html>
```

OUTPUT:



Explanation:

- <!DOCTYPE html> declares the HTML5 document type.
- <meta charset="UTF-8"> supports a wide range of characters.
- The viewport meta tag helps the webpage adapt to different screen sizes.
- Properly closed tags prevent structural errors.
- Semantic elements organize the content clearly.

Output: The browser displays a Welcome heading, an About Us section with a description, and a footer containing copyright information.

5. Suggest Techniques to Ensure Cross-Browser Compatibility

The following techniques help websites display consistently across different browsers.

1. **Use Valid HTML:** Close all tags correctly and follow HTML5 standards to avoid structural errors.
2. **Use CSS Reset:** Normalize default browser styles to reduce differences in margins, padding, and fonts.
3. **Test Multiple Browsers:** Check the webpage in Chrome, Firefox, Microsoft Edge, and Safari.
4. **Use Responsive Design:** Apply CSS media queries to make the website suitable for desktops, tablets, and mobile devices.

5. **Validate HTML:** Use the W3C Markup Validation Service to identify missing tags and invalid elements.
6. **Avoid Browser-Specific Features:** Prefer standard HTML and CSS properties supported by modern browsers.
7. **Maintain Consistent Styling:** Use appropriate fonts, spacing, colors, and layouts to achieve a uniform appearance.

3. Form Submission Failure

1. Identify Issues in the Form Configuration

The given HTML form contains the following issues:

- Incorrect Button Type: The button uses type="button", which does not submit the form automatically.
- GET Method: The GET method sends form data through the URL, which is unsuitable for passwords.
- Missing Labels: The form does not contain labels for the username and password fields.
- Missing Validation: The input fields do not use the required attribute.

These issues can affect form submission, usability, and security.

2. Analyze Why the HTTP Request Is Not Triggered

The HTTP request is not triggered because the button has type="button" instead of type="submit".
A button with type="button" does not submit the form automatically. Therefore, clicking Login will not send the form data to the server unless JavaScript is used to handle the submission.
Changing the button type to submit allows the browser to submit the form.

3. Interpret the Role of GET Method in This Scenario

The GET method sends form data as query parameters in the URL.
Disadvantages of GET:

- Passwords may appear in browser history and server logs.
- URLs containing credentials may be stored or shared.

- GET is not appropriate for transmitting sensitive login information.

Therefore, the POST method should be used for login forms, along with HTTPS for secure transmission.

4. Propose a Corrected Implementation

The following HTML code corrects the form submission issue and improves usability.

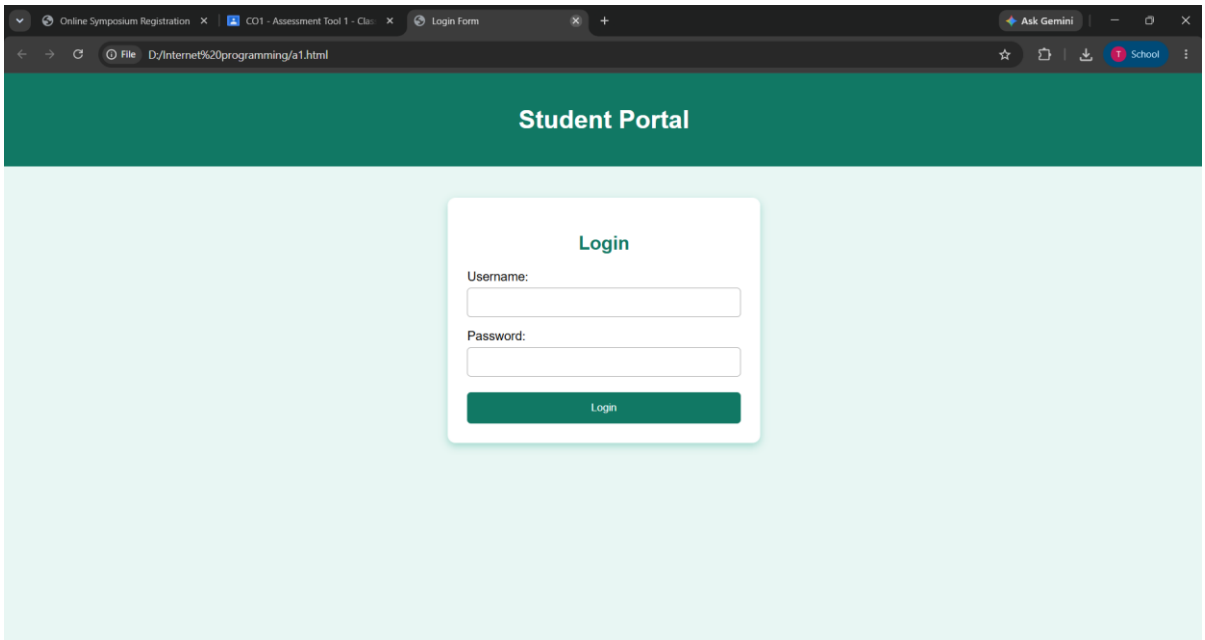
HTML + CSS Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #e8f6f3;
      margin: 0;
      padding: 0;    }
    header {
      background-color: #117864;
      color: white;
      text-align: center;
      padding: 20px;  }
    main {
      max-width: 350px;
      margin: 40px auto;
      padding: 25px;
      background-color: white;
      border-radius: 10px;
      box-shadow: 0 4px 10px #a2d9ce; }
    h2 {
```

```
            text-align: center;
            color: #117864; }
        label {
            display: block;
            margin-top: 15px;  }
        input {
            width: 100%;
            padding: 10px;
            margin-top: 5px;
            box-sizing: border-box;
            border: 1px solid #aaa;
            border-radius: 5px;  }
        button {
            width: 100%;
            padding: 12px;
            margin-top: 20px;
            background-color: #117864;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer; }
        button:hover {
            background-color: #0b5345;}
    </style>
</head>
<body>
    <header>
        <h1>Student Portal</h1>
    </header>
    <main>
        <h2>Login</h2>
```

```
 <form action="/submit" method="POST">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required>
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required>
        <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

OUTPUT:



Explanation:

- method="POST" sends the form data in the HTTP request body.
- type="submit" triggers form submission.
- type="password" hides the password characters on the screen.
- required ensures that both fields are filled before submission.
- CSS improves the appearance and readability of the login form.

Output: A login webpage displaying username and password fields with a green Login button.

5. Suggest Improvements for Secure Data Transmission

1. **Use HTTPS:** Encrypts data transmitted between the browser and server.
2. **Use POST:** Keeps credentials out of the URL query string.
3. **Validate Input:** Check submitted data on both the client and server.
4. **Hash Passwords:** Store passwords using secure password-hashing algorithms such as Argon2id or bcrypt.
5. **Use Secure Sessions:** Protect session cookies with Secure, HttpOnly, and appropriate SameSite settings.
6. **Avoid Exposing Credentials:** Never store passwords in URLs or display them in error messages.